

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

CO5 ASSESSMENT TOOL 1 – Research Paper Review / Literature Survey

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO5	Assessment:	Assessment Tool 1 – Research Paper Review / Literature Survey
Weightage:	20%	Total Marks:	25
CO5 Statement:	Evaluate emerging technologies and societal impacts, maintaining and evolving software systems responsibly		
Student Name:	K v charan teja	Reg. No.:	192424167
Date:		Duration:	60 minutes

Code of Conduct

I k v charan teja (192424167) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Research Paper Review / Literature Survey

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **conduct a literature survey and review relevant research papers** to understand the existing approaches, techniques, technologies, limitations, and research gaps related to the problem.

The literature survey should include:

1. Identify and select **relevant research papers** related to the assigned problem statement from reputed journals, conferences, or scholarly databases.
2. Summarize the **objectives, methodologies, technologies, datasets, and major findings** of the selected research papers.
3. Compare the existing approaches based on suitable parameters such as **accuracy, performance, scalability, efficiency, methodology, or other problem-specific metrics**.
4. Identify the **limitations and challenges** of the existing research approaches.
5. Analyze the literature to identify the **research gap** or unresolved issues related to the assigned problem.
6. Discuss the **possible improvements or future research directions** based on the identified research gap.
7. Prepare a **comparative literature survey table** containing details such as author/year, research problem, methodology, dataset/tools, key findings, limitations, and research gap.
8. Provide proper **citations and references** using a consistent referencing style.

Deliverable: Submit a **Research Paper Review / Literature Survey Report** containing the selected research papers, individual paper reviews, comparative analysis, identified research gaps, future directions, and properly formatted references.

Research Paper Review / Literature Survey

Title: AI-Based Software Defect Prediction Using Machine Learning

1. Introduction

Software defects are one of the major challenges in software development. Defects can reduce software quality, increase maintenance costs, cause project delays, and affect user satisfaction. Traditional software testing is important, but testing every module with the same level of effort can consume significant time and resources.

Software Defect Prediction (SDP) uses historical software data and Artificial Intelligence techniques to identify modules, classes, or components that are more likely to contain defects. Machine Learning algorithms can analyze software metrics such as code complexity, size, coupling, cohesion, and historical defect information to predict fault-prone components.

Recent research has explored traditional machine learning, ensemble learning, deep learning, and AI-based approaches for software defect prediction. Commonly used datasets include NASA and PROMISE datasets, while frequently used evaluation metrics include Accuracy, Precision, Recall, F1-Score, and AUC.

The purpose of this literature survey is to review existing research on AI-based software defect prediction, compare different approaches, identify their limitations, and determine important research gaps and future directions.

2. Objectives of the Literature Survey

The main objectives of this literature survey are:

1. To study existing Artificial Intelligence and Machine Learning approaches used for software defect prediction.
2. To review the methodologies, datasets, tools, and findings of selected research studies.
3. To compare traditional machine learning and deep learning approaches.
4. To identify the limitations and challenges in existing software defect prediction systems.
5. To identify research gaps and propose possible future improvements.

3. Selected Research Paper Reviews

Paper 1: Software Defect Prediction Using Supervised Machine Learning Techniques: A Systematic Literature Review

Authors: F. Matloob et al.

Year: 2021

Objective

The objective of this study was to review the use of supervised machine learning techniques for software defect prediction and identify important research trends in software defect classification.

Methodology

The researchers followed a systematic literature review methodology. A large number of research papers were initially collected from scholarly databases, followed by a selection process to identify the most relevant studies for detailed review.

The reviewed studies focused on supervised learning techniques for classifying software modules as defective or non-defective.

Technologies and Approaches

The study examined techniques such as:

- Decision Trees
- Support Vector Machines
- Naive Bayes
- Random Forest
- Neural Networks
- Other supervised machine learning classifiers

Dataset and Tools

The reviewed studies included commonly used public software defect datasets and machine learning-based classification approaches.

Major Findings

The study showed that machine learning can support software quality assurance by identifying potentially defective modules before software delivery. Different algorithms perform differently depending on the dataset, software metrics, preprocessing techniques, and evaluation methods.

Limitations

- Results are dependent on the quality of available datasets.
- Different studies use different evaluation methods, making direct comparison difficult.
- Many models focus mainly on binary classification.
- Practical deployment in real industrial environments requires further investigation.

Paper 2: A Systematic Literature Review on Software Defect Prediction Using Artificial Intelligence: Datasets, Data Validation Methods, Approaches, and Tools

Authors: Jalaj Pachouly et al.

Year: 2022

Objective

The objective of this research was to provide a comprehensive review of Artificial Intelligence techniques used for software defect prediction, with particular attention to datasets, data validation methods, prediction approaches, and available tools.

Methodology

The researchers conducted a systematic literature review based on established literature review guidelines. The study analyzed 146 research studies related to software defect prediction.

Technologies and Approaches

The paper reviewed:

- Machine Learning
- Deep Learning
- Artificial Neural Networks
- Ensemble Learning
- Cross-Project Defect Prediction
- Within-Project Defect Prediction
- Cross-Version Defect Prediction

Dataset and Tools

The research discussed commonly used datasets such as:

- NASA datasets
- PROMISE repository datasets
- Publicly available software project datasets

Major Findings

The study found that many traditional datasets have limited features and labels. It also identified increasing interest in Deep Learning and Artificial Neural Networks. The authors highlighted the need to improve datasets, data validation, and prediction outcomes so that results become more useful to software development teams.

Limitations

- Many existing datasets contain limited defect-related information.

- Binary classification does not provide detailed information about defect types.
 - Data validation methods are not consistently applied.
 - Many research models do not provide directly actionable results for developers.
-

Paper 3: Software Fault Prediction Using Data Mining, Machine Learning and Deep Learning Techniques: A Systematic Literature Review

Year: 2022

Objective

The purpose of this research was to systematically study the use of Data Mining, Machine Learning, and Deep Learning techniques for software fault prediction.

Methodology

The researchers used a systematic literature review process and organized the review into planning, conducting, and reporting stages.

Technologies and Approaches

The study compared three major categories:

- Data Mining techniques
- Machine Learning techniques
- Deep Learning techniques

Dataset and Tools

The research identified NASA and PROMISE as widely used repositories. Common software metrics included:

- CK metrics
- McCabe complexity metrics
- Halstead metrics

Major Findings

The research showed that software fault prediction has evolved from traditional data mining approaches toward Machine Learning and Deep Learning techniques. Common evaluation metrics include Accuracy, Precision, Recall, F1-Score, and AUC.

Limitations

- Results can vary significantly across datasets.
- Public datasets may not represent real industrial software systems.
- Class imbalance can affect prediction performance.

- Complex Deep Learning models may require higher computational resources.
-

Paper 4: Application of Deep Learning in Software Defect Prediction: Systematic Literature Review and Meta-analysis

Year: 2023

Objective

The objective of this study was to investigate the application of Deep Learning in software defect prediction and compare Deep Learning approaches with traditional Machine Learning approaches.

Methodology

The researchers performed a systematic literature review and meta-analysis. Sixty-three primary studies passed the literature review quality evaluation, while nineteen studies met the criteria for meta-analysis.

Technologies and Approaches

The study reviewed Deep Learning techniques including:

- Convolutional Neural Networks
- Deep Neural Networks
- Long Short-Term Memory networks
- Deep Belief Networks
- Stacked Denoising Autoencoders

Dataset and Tools

PROMISE and NASA datasets were among the most frequently used datasets for training and testing Deep Learning models.

Major Findings

The meta-analysis found that Deep Learning was generally favored over traditional Machine Learning in terms of Accuracy, F-measure, and AUC across the analyzed studies and datasets. However, the effectiveness of Deep Learning depends on proper optimization of performance-related parameters.

Limitations

- Deep Learning requires more computational resources.
 - Model performance depends heavily on parameter optimization.
 - Limited high-quality labeled datasets are available.
 - Deep Learning models can be difficult to interpret.
-

Paper 5: Machine Learning in Software Defect Prediction: A Business-Driven Systematic Mapping Study

Year: 2023

Objective

The objective of this study was to examine software defect prediction research from a business and industrial adoption perspective.

Methodology

The researchers conducted a systematic mapping study using a large collection of primary research studies. After applying selection criteria, 742 primary studies were analyzed.

Technologies and Approaches

The study examined trends involving:

- Decision Trees
- Neural Networks
- Support Vector Machines
- Continuous Integration
- Just-in-Time Defect Prediction
- Cost-Effective Defect Prediction

Dataset and Tools

The research found that NASA and open-source datasets are widely used, while commercial and real-world industrial datasets are comparatively less common.

Major Findings

The study concluded that although Machine Learning-based software defect prediction has received significant research attention, business applicability and industrial adoption remain important challenges. More practical and in-vivo studies are required.

Limitations

- Academic datasets dominate the research.
- Real industrial validation is limited.
- Business cost and practical usefulness are not always considered.
- A highly accurate model may not automatically provide the greatest business value.

4. Comparative Literature Survey Table

Author / Year	Research Problem	Methodology	Dataset / Tools	Key Findings	Limitations	Research Gap
Matloob et al., 2021	Software defect classification	Systematic literature review of supervised ML techniques	Public defect datasets and ML classifiers	ML can support early identification of defective modules	Dataset dependency and inconsistent evaluation	Need for standardized comparison and practical validation
Pachouly et al., 2022	AI-based software defect prediction	Systematic literature review of 146 studies	NASA, PROMISE and AI approaches	AI and DL are increasingly important for defect prediction	Limited dataset features and binary labels	Need for improved datasets and multi-label prediction
Systematic Review, 2022	Comparison of DM, ML and DL for fault prediction	Systematic literature review	NASA, PROMISE, CK, McCabe and Halstead metrics	ML and DL are widely used and evaluated using common metrics	Dataset quality and class imbalance	Need for more robust and realistic datasets
Deep Learning Meta-analysis, 2023	Deep Learning for defect prediction	SLR and meta-analysis	NASA and PROMISE datasets	DL showed advantages in several performance measures	High computational cost and low interpretability	Need for explainable and efficient DL models
Business-Driven Mapping Study, 2023	Practical adoption of ML defect prediction	Systematic mapping study	Academic, NASA and open-source datasets	Business adoption requires more practical validation	Limited commercial and industrial datasets	Need for real-world, cost-aware prediction systems

The selected literature shows that methodology, dataset selection, validation, performance metrics, and practical applicability are important parameters for comparing existing approaches.

5. Comparative Analysis of Existing Approaches

5.1 Traditional Machine Learning

Traditional Machine Learning techniques such as Random Forest, Decision Trees, Support Vector Machines, and Naive Bayes are commonly used for software defect prediction.

Advantages

- Easier to implement.
- Requires comparatively fewer computational resources.
- Can perform effectively with structured software metrics.
- Some models provide better interpretability than complex Deep Learning systems.

Disadvantages

- Performance depends strongly on feature selection.
 - May struggle with highly complex data.
 - Dataset imbalance can significantly affect results.
-

5.2 Ensemble Learning

Ensemble methods combine multiple models to improve prediction performance.

Advantages

- Can improve prediction stability.
- Often performs well with structured datasets.
- Can reduce weaknesses of individual classifiers.

Disadvantages

- Increased model complexity.
 - May require parameter tuning.
 - Higher computational requirements than individual simple models.
-

5.3 Deep Learning

Deep Learning approaches include CNN, DNN, LSTM, DBN, and other neural network architectures.

Advantages

- Can learn complex patterns automatically.
- Shows promising results in Accuracy, F-measure, and AUC in reviewed research.
- Can potentially process more complex software-related information.

Disadvantages

- Requires more computational resources.
 - Requires careful parameter optimization.
 - Often lacks interpretability.
 - Performance can be affected by limited or poor-quality training data.
-

6. Comparison Based on Important Parameters

Parameter	Traditional ML	Ensemble Learning	Deep Learning
Accuracy	Good	High in many datasets	Potentially high
Computational Cost	Low to Medium	Medium	High
Interpretability	Relatively high	Medium	Low
Data Requirement	Moderate	Moderate	Often high
Feature Engineering	Usually required	Usually required	Can learn complex features
Scalability	Good	Good	Depends on infrastructure
Implementation Complexity	Low to Medium	Medium	High

Analysis: No single approach is universally best. Model performance depends on dataset quality, class imbalance, feature representation, validation strategy, and the practical requirements of the software project. Existing reviews also show that data quality issues such as dimensionality, class imbalance, class overlap, and missing data can significantly influence software fault prediction performance.

7. Limitations and Challenges in Existing Research

Based on the reviewed literature, the major challenges are:

1. Dataset Quality

Many software defect prediction models depend heavily on public datasets. These datasets may contain missing information, limited features, class imbalance, or insufficient defect labels.

2. Class Imbalance

In many datasets, non-defective modules greatly outnumber defective modules. This can lead to biased models.

3. Limited Industrial Validation

A significant amount of research uses NASA, PROMISE, and open-source datasets, while fewer studies use commercial or real industrial data.

4. Lack of Standardized Evaluation

Different researchers use different datasets, validation methods, preprocessing techniques, and performance metrics. Therefore, direct comparison between models can be difficult.

5. Limited Explainability

Complex AI and Deep Learning models may provide accurate predictions without clearly explaining why a particular software module is predicted as defective.

6. Binary Classification

Many existing systems only predict whether a module is defective or non-defective. They may not provide information about defect type, severity, or recommended corrective action.

7. Business and Cost Considerations

High predictive accuracy does not always mean that a system provides the highest practical value. Testing effort, maintenance cost, and business impact should also be considered.

8. Identified Research Gap

From the literature survey, the following research gaps are identified:

1. **Most existing studies rely heavily on public datasets**, which may not represent the characteristics of real industrial software projects.
2. **Many models focus only on binary defect prediction** and do not provide detailed information about defect severity or defect categories.
3. **Data quality and class imbalance remain major challenges**, and there is no single preprocessing strategy suitable for all datasets.
4. **Deep Learning models may provide improved performance but often lack explainability**, making them difficult for software engineers to trust and use.
5. **Business and cost-aware evaluation is limited** in many research studies.
6. **Most systems only predict defects and do not provide actionable recommendations**, such as which part of the code should be tested first or what type of defect may occur.

Therefore, there is a need for an **intelligent, explainable, cost-aware, and practical software defect prediction system that can use high-quality data and provide actionable information to software development teams**.

9. Proposed Future Research Directions

Based on the identified research gaps, the following future directions are proposed:

1. Explainable AI for Defect Prediction

Future systems should not only predict defects but also explain the important factors responsible for the prediction.

2. Multi-Label Defect Prediction

Instead of simply predicting defective or non-defective modules, future research can predict multiple defect categories and severity levels.

3. Real-World Industrial Datasets

Researchers should validate models using real-world software projects and industrial datasets to improve practical applicability.

4. Hybrid Machine Learning and Deep Learning Models

Hybrid approaches can combine the interpretability of traditional Machine Learning with the pattern-learning capabilities of Deep Learning.

5. Cost-Aware Prediction

Future systems should consider testing cost, development resources, and business impact rather than focusing only on Accuracy.

6. Integration with Continuous Integration and DevOps

Defect prediction models can be integrated into CI/CD pipelines to provide continuous prediction during software development.

7. Automated Data Quality Improvement

Future research can develop automated methods for detecting missing values, class imbalance, overlapping classes, and other dataset quality issues before model training.

These directions are consistent with recent literature identifying the need for better datasets, practical adoption, data-quality handling, and more useful prediction systems.

10. Conclusion

This literature survey reviewed research on **AI-Based Software Defect Prediction Using Machine Learning**. The selected studies demonstrate that Machine Learning, Ensemble Learning, and Deep Learning can support early identification of potentially defective software components.

Traditional Machine Learning approaches remain useful because of their simplicity and relatively lower computational requirements. Deep Learning has shown promising predictive performance in several studies, but it also introduces challenges related to computational cost, data requirements, and explainability.

The literature also reveals several important limitations, including poor dataset quality, class imbalance, lack of standardized evaluation, limited industrial validation, and insufficient focus on business value.

The major research gap identified is the need for an **explainable, cost-aware, multi-label, and industry-oriented software defect prediction system**. Future research should focus on integrating AI-based defect prediction into real software development environments and providing actionable recommendations to developers rather than only generating a defective or non-defective classification.

11. References

1. Matloob, F. et al. (2021). *Software Defect Prediction Using Supervised Machine Learning Techniques: A Systematic Literature Review*. Intelligent Automation & Soft Computing.
2. Pachouly, J., Ahirrao, S., Kotecha, K., Selvachandran, G., & Abraham, A. (2022). *A Systematic Literature Review on Software Defect Prediction Using Artificial Intelligence: Datasets, Data Validation Methods, Approaches, and Tools*. Engineering Applications of Artificial Intelligence.
3. *Software Fault Prediction Using Data Mining, Machine Learning and Deep Learning Techniques: A Systematic Literature Review*. (2022). Computers and Electrical Engineering.
4. *Application of Deep Learning in Software Defect Prediction: Systematic Literature Review and Meta-analysis*. (2023). Information and Software Technology.
5. *Machine Learning in Software Defect Prediction: A Business-Driven Systematic Mapping Study*. (2023). Information and Software Technology.
6. Bhandari, K., Kumar, K., & Sangal, A. L. (2023). *Data Quality Issues in Software Fault Prediction: A Systematic Literature Review*. Artificial Intelligence Review.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Selection & Review of Research Papers(25%)	Selects highly relevant, recent, credible papers and provides comprehensive reviews of objectives, methodology, tools, datasets, and findings	Selects relevant papers and provides good reviews with minor gaps	Selects moderately relevant papers with basic reviews	Papers are irrelevant or reviews are incomplete
Comparative Analysis(20%)	Provides a detailed comparison of methodologies, tools, datasets, results, and performance using a clear comparison table	Provides a good comparison with relevant parameters	Provides limited comparison with few parameters	Little or no meaningful comparison
Critical Analysis & Research Gap(25%)	Clearly analyzes limitations and identifies significant,	Identifies relevant limitations and research gaps	Identifies basic gaps with limited analysis	Research gaps are unclear or missing

	well-supported research gaps			
Future Directions & Relevance(15 %)	Proposes practical and innovative future directions directly relevant to the assigned problem statement	Proposes relevant future directions	Provides general future suggestions	Future directions are missing or unrelated
Documentation , Citations & References(15 %)	Excellent academic structure, clear presentation, accurate citations, and consistent referencing	Well-organized with minor formatting/citation issues	Adequately documented with some errors	Poorly organized with inadequate or incorrect references

Criteria	Mark
Selection & Review of Research Papers(25%)	/5
Comparative Analysis(20%)	/5
Critical Analysis & Research Gap(25%)	/5
Future Directions & Relevance(15%)	/5
Documentation, Citations & References(15%)	/5
TOTAL	/25